PEKING UNIVERSITY / COURSE NOTES

第 2 课
SDD 与 CampusClaw 规约
完整要求整理

包含课程全流程要求、环境和操作指南，以及你要求补入的
Review 对照 · 终稿全文（Ex.1—Ex.4，按原 PPT 10 个版面收录）。

4	10	7 / 20 / 22
份原文 Markdown	个 Ex 版面与点评	Requirement / Scenario / tasks

阅读顺序
第一部分：课程目标、SDD、OpenSpec、建仓、Propose、需求、Review 和交付核对。
第二部分：Ex.1 proposal → Ex.2 design（4 段）→ Ex.3 spec（3 段）→ Ex.4 tasks（2 段）。
第三部分：4.6—4.9 Review 原文、AI 提示词及 OpenSpec 命令附录。

原始课件：https://devops.hello1023.com/课件/第2课-课件-SDD与CampusClaw规约/index.html

CONTENTS / 目录

目录与核对范围

第一部分 课程要求与指导：SDD、课表教学案例、OpenSpec、学期仓与当堂交付

第二部分 Review 对照 · 终稿全文：完整的 Ex.1—Ex.4

Ex.1 proposal.md：Why / What / Non-goals

Ex.2 design.md（上）：Context 与 Goals

Ex.2 design.md（中）：会话、栈与班级隔离

Ex.2 design.md（下）：上传入库数据流

Ex.2 design.md（部署）：Docker Compose 与 /health

Ex.3 spec.md（上）：登录与角色

Ex.3 spec.md（中）：隔离与上传

Ex.3 spec.md（下）：预置、安全与部署

Ex.4 tasks.md（上）：第 1–4 节

Ex.4 tasks.md（下）：第 5–7 节

第三部分 4.6、4.7、4.8、4.9 Review 原文与命令附录

原文收录说明：本册按你提供并对照课件核验的 Ex.1—Ex.4 正文排版，保留原有 Requirement、Scenario、任务、英文关键词和标点。每页“点评”在对应正文之后单列；不会混入四份 Markdown。另附独立可编辑的四份 Markdown 压缩包。

值得注意：课件原文在“缺少会话密钥时”这一 Scenario 中出现“应用启动 MUST 失败或使用文档明确禁止的不安全默认值”的措辞，读起来可能有歧义。本册正文忠实保留，文末另作提示，不静默改写。

PART I / COURSE GUIDE

第一部分 · 完整要求与操作指导

第 2 课：SDD 与 CampusClaw 规约——完整要求整理

> 学习路线：理解 SDD → 课表案例练习 → OpenSpec 入门 → 新建 CampusClaw 学期仓 → Propose → Review → 新对话验收。本文将指导性内容与 PPT 中的 Ex.1—Ex.4 终稿原文分开排版，避免将教学示例与本课必交作业混淆。

1. 课程目标与本课边界

本课正式交付：新建 CampusClaw 独立学期仓，初始化 OpenSpec，完成 README 三行说明与项目规则文件；创建一个 change，形成 `proposal.md`、`design.md`、`specs/auth-upload/spec.md`、`tasks.md` 四份 Markdown；进行互相 Review，并用新 AI 对话验证规约能被独立读懂。

本课“三不”：不 Apply、不编写业务页面、不运行真实 Docker Compose。Dockerfile、数据库建表、登录路由、上传接口等是第 3 课根据 tasks 实施的内容，本课只明确它们未来必须满足的规格。课表练习仓不能混入 CampusClaw 学期仓；口头承诺不等于写入仓库。

2. SDD（规约驱动开发）的方法

- 先澄清需要做的功能与非目标，再决定技术栈、数据模型和关键数据流。
- 用 Requirement + Scenario 表达能够判定通过或失败的行为；正向用例之外，还必须覆盖错误密码、未登录、学生越权和跨班等负向场景。
- 将规则和规约保存在仓库而不是仅保存在聊天记录，必要时通过 Git 管理变更。
- 让任务清单覆盖全部需求，每条任务提供具体 `verify`，后续按规约进行实现和验证。
- 课表示范指出，若只让 AI 快速完成网页，数据库是否持久化、教室字段、刷新更新和登录边界等容易遗漏；第二轮口头补充会引起返工。

课表教学案例（非 CampusClaw 交付）

课表示范 change 名为 `add-weekly-class-schedule`，示范路线为 Flask + SQLite + Jinja2。样例能力包括匿名访问每周课程页面、课程数据存 SQLite、星期和节次布局、可选教室字段，以及直接修改数据库后刷新页面能看到更新。样例明确排除登录、教务系统对接、换课提醒、课程管理后台等范围。它用四件套和 `openspec validate ... --strict` 展示如何把含糊需求转化为可验证规约。

3. OpenSpec 工作流、文件与命令

阶段	作用	本课状态
Explore	研究已有仓库、发现问题，可选	可选
Propose	创建 change，生成四件套初稿	必须完成

阶段	作用	本课状态
Review	检查范围、设计、Scenario 和任务覆盖	必须完成
Apply	根据 tasks 编写和测试业务代码	本课禁止，第 3 课
Archive	将成熟规约归档为主规格	本课禁止，第 3 课

四件套的职责：`proposal.md` 说明为什么、做什么与不做什么；`design.md` 说明技术路线与关键数据流；`spec.md` 用 Requirement / Scenario 确定行为；`tasks.md` 指出实现顺序和 verify。规则文件（如 `AGENTS.md`）用于长期约束，技能封装流程，斜杠命令触发对应流程。

4. 环境、建仓与初始化

- 编程工具：Trae、Cursor、Claude Code、Codex、WorkBuddy 任选一种；Node.js 应为 v20 或更高版本。
- 安装 OpenSpec CLI：`npm install -g @fission-ai/openspec@latest`，用 `openspec --version` 验证；不能全局安装时可考虑 npx 方式。
- 自己在系统终端建 CampusClaw 空仓并执行 `git init`；不要让 AI 在对话中代建仓。与课表练习仓保持分离。
- README 写三行：价值主张、核心使用场景、本学期不做什么。仓根添加项目规则文件，明确“无规约不写代码”。
- 用编程工具打开自己的学期仓根，执行 `openspec init --tools cursor`（将 cursor 替换成所使用的工具），重启工具并确认 OpenSpec 斜杠命令可用。
- 检查是否有 `openspec/config.yaml`、`openspec/changes/`、README 和规则文件。

典型终端命令

```
node -v
npm install -g @fission-ai/openspec@latest
openspec --version
mkdir -p ~/campusclaw-mygroup
cd ~/campusclaw-mygroup
git init
openspec init --tools cursor
```

5. Propose：生成并核对 CampusClaw change

在新的 AI 对话中指向正确工作区，输入 `/opsx-propose`，描述登录、角色权限、班级隔离和教师上传知识库。明确：教师学生账号密码登录；未登录受保护页面需跳转；学生上传被拒；A 班不得越权访问 B 班；教师上传后解析入库，本班列表从数据库展示；密码哈希，密钥取自环境变量；标准部署规划为 Compose，提供 `/health`。本次不做检索问答、AI 对话、作业提交批改、注册找回密码。

等四件套生成完，记下实际 change 名称。初稿完成后检查 `proposal` 是否含 Why/What/Capabilities/Non-goals；`spec` 是否含七类 Requirement 和可判定 Scenario；`design` 是否含技术栈、会话与密码、服务端班级隔离、上传链路及部署规划；`tasks` 是否按实施顺序分节且每条含 verify。发现缺漏时，新建对话并只修改 change 下 Markdown，让 AI 列明改动文件和补充内容。

6. CampusClaw 的七类核心需求

主题	核心要求	必须覆盖的负向或持久化验证
用户登录	教师、学生密码登录，建立带用户 ID、角色和班级信息的会话	错误密码不建会话；未登录页面跳转、API 401
角色权限	教师上传本班材料；学生只读	学生直接 POST 上传 403 且无新增 DB / 文件
班级隔离	集合查询与按 ID 访问均按会话中的 class_id 在服务端校验	A 班不得访问 B 班单项和列表
上传与入库	支持 txt/md，保存文件、解析文本、双表事务写入，列表刷新可见	非法/空/解析失败不留脏数据；A 班学生上传后可只读查看
预置核心数据	两班、三类样本账号、两班不同材料，以及班级/用户/讲义/作业/助手/技能结构	初始数据可查询、可分辨两班
哈希与密钥	使用密码哈希，SECRET_KEY 由服务端环境变量注入	DB 无明文；缺密钥不能静默使用不安全默认值
Compose 与 health	数据和上传卷持久化、登录页可访问、匿名健康检查	down/up 后原数据在；health 不依赖登录

PPT 的终稿示例共有 7 条 Requirement、20 条 Scenario。Ex.4 的实施规划分为 T1–T7、22 个未勾选任务。这些数字属于老师示例的覆盖度，不代表本课须在课堂上运行 22 项业务实现。

7. 建议的技术方案（来自 Ex.2）

- Flask 3.x 负责登录页、材料列表、上传和 JSON API；SQLite 3 默认使用单文件 `data/app.db`；Compose 内推荐 Gunicorn。
- 采用签名 Cookie；保留用户、角色和班级信息，设置 HttpOnly 和 SameSite=Lax；`SECRET_KEY` 由环境变量提供。
- 服务端统一保护受限路由：没有会话时页面重定向 `/login`、API 返回 401；学生上传 403。
- 列表查询强制包含 `WHERE class_id = :session_class_id`；按 ID 访问同样校验班级；跨班返回 403 或 404 中的一种并在 README 固定。
- 上传 `multipart/form-data` 时按会话身份授权，校验 txt/md，保存到按班级划分的上传目录；解析失败删除文件；成功后在一个事务中写入 `materials` 和 `knowledge_entries`。
- Dockerfile、Compose、`.env.example`、数据和 uploads 卷、自动建表种子、无认证的 `/health` 都需要在本课 design 和 tasks 中规划，但代码留到第 3 课。

8. Review、交叉检查与新对话验收

- Review 只修改 Markdown：以 README 和 proposal 看清项目范围；以 design 看清会话、权限、隔离、上传及部署；以 spec 检查能否客观判定；以 tasks 核实每项 Requirement 都有任务和 verify。
- 与同学交换文件，检查对方能否从文件中准确说明“做什么、不做什么”；至少核对跨班拒绝、学生 403 和教师上传入库三项场景。

- 在新会话中，不粘贴以前的聊天记录，仅让 AI 读取当前仓库的 OpenSpec、README 和项目规则文件；让它给出至少三条硬性要求和文件路径，并回答 A 班学生能否看 B 班，以及上传后知识库/材料列表如何变化，必须附原文依据。
- 第 2 课的 tasks 均保持 `[ ]`，不 Apply、不编写业务页面、不运行真实 Compose。后续实施阶段可根据环境对照验收，并在合适阶段运行 `openspec validate <change> --strict`。

9. 本课交付核对表

- ☐ CampusClaw 独立学期仓建立并完成 `git init`。
- ☐ README 三行说明与项目规则文件就位。
- ☐ OpenSpec 初始化正确，config.yaml 与 changes/ 目录存在。
- ☐ change 的 proposal、design、spec、tasks 四件套完整。
- ☐ 完整覆盖登录、角色、隔离、上传、预置、安全和部署；关键 Scenario 可直接判定。
- ☐ tasks 与全部 Requirement 对应，每条含 verify，所有 checkbox 为 `[ ]`。
- ☐ 与相邻同学互查并修改规约。
- ☐ 新 AI 对话能仅凭仓库引用 spec 原文，准确回答关键问题。
- ☐ 没有提前 Apply、编写业务页面或运行真实 Compose；课表演示不混入本项目。

10. 阅读后半册的方式

后半册标题为 “Review 对照 · 终稿全文”，按照 PPT 的 Ex.1、Ex.2（上/中/下/部署）、Ex.3（上/中/下）、Ex.4（上/下）原顺序排版。四份 Markdown 的措辞、任务 checkbox 与 Scenario 均按你提供的原文保留；每一个 PPT 版面末尾另列老师的 “点评”，不将点评混入 Markdown 文件。接着收录 4.6–4.9 Review 页面与 OpenSpec 命令附录。

PART II / VERBATIM EXEMPLARS

Review 对照 · 终稿全文

以下依次呈现你要求保留的 Ex.1 proposal、Ex.2 design（上 / 中 / 下 / 部署）、Ex.3 spec（上 / 中 / 下）及 Ex.4 tasks（上 / 下），每段附课堂点评。原文仅对 Markdown 进行版式渲染，不改动措辞。

课程提醒
与相邻同学互查时，重点问两句：Scenario 能不能判定通过？tasks 有没有覆盖全部 Requirement？本课所有 tasks 仍为 `[ ]`，后续 Apply 才能逐项勾选。

EXEMPLAR 01 / 10 · Ex.1

终稿 · proposal.md

Why / What / Non-goals · 原文全文（Markdown 格式排版）

Why

CampusClaw 是面向中小学的教研智能体，覆盖学情分析、备课、命题、课堂与教研协同，教研材料与知识库是其能力底座。当前材料分散在群文件与网盘中，教师难以检索复用，班级之间也缺少数据边界。本变更先建立身份、角色与班级隔离，让教师登录后上传并管理本班材料，学生仅可查看本班内容，为后续检索与智能问答打基础。

What Changes

- 用户登录：教师、学生两类角色；账号密码登录；未登录访问受保护页面时引导至登录页。
- 角色权限：教师可上传与管理本班材料；学生对本班材料只读；学生调用上传接口 MUST 被拒绝（如 HTTP 403）。
- 班级隔离：班级为数据边界；所有业务查询在服务端按会话中的班级过滤；A 班用户不得读取 B 班材料（含直接访问与列表越权）。
- 材料上传与知识库入库：教师上传后解析并写入知识库，材料关联本班；本班材料列表查库展示，上传后可见新记录。
- 预置样本数据：班级 A/B、教师 A、学生 A1/B1，以及两班可区分的材料标题；建立班级、用户、讲义、作业、助手、技能六类核心数据结构并写入种子数据。
- 安全与运行：密码 MUST 哈希存储，禁止明文；会话/签名密钥仅通过服务端环境变量注入；标准启动方式为 Docker Compose；应用提供 `GET /health`。
- 文档：README 三行说明（价值、场景、不做）与 Docker 启动方式；新增 `docker-compose.yml`、`.env.example`。

Capabilities

New Capabilities

- `auth-upload`：登录与会话、按角色授权、按班级隔离、教师上传材料入库、预置数据、密码哈希与会话密钥、Docker Compose 与 `/health` 等可验收行为。

Modified Capabilities

（无——`openspec/specs/` 尚无主规格。）

Impact

- 新建 Web 应用与 API（登录、材料列表、教师上传、健康检查）。
- 新建 SQLite（或等价）持久层：用户、班级、材料、知识库条目及六类域表与种子数据脚本。
- 项目根增加 `Dockerfile`、`docker-compose.yml`、`.env.example`；依赖清单（如 `requirements.txt`）。
- 可运行实现按 `tasks.md` 在 Apply 阶段完成；本 change 以规约四件套为交付物。

Non-goals（非目标）

本课/本 change 明确不做以下内容（避免范围膨胀；表结构可预留字段，但不验收）：

- 检索与问答：不做 RAG、全文检索、智能问答、基于知识库的对话。
- 对话助手：不做聊天界面、Agent 编排、技能包运行时。
- 作业流程：不做作业提交、批改、成绩、审计与错题本。
- 身份与企业集成：不做多校/多租户、OAuth/OIDC、SSO、短信/邮箱验证码登录。
- 生产运维：不做 K8s、多副本高可用、完整 CI/CD 流水线；标准启动为 Docker Compose 本地/演示环境即可。
- 前端替代服务端安全：不以隐藏按钮、禁用表单作为权限或班级隔离的唯一手段。

> **PPT 点评**
>
> Why 只讲痛点；What 列交付能力；Non-goals 分条写清“检索、对话、作业、SSO、生产 HA”——这是防范围膨胀的第一道闸。

EXEMPLAR 02 / 10 · Ex.2

终稿 · design.md（上）

Context 与 Goals · 原文全文（Markdown 格式排版）

Context

仓库根已有 README 三行说明与 OpenSpec change 四件套；尚无业务应用代码。动机见 `proposal.md`；可验收行为见 `specs/auth-upload/spec.md`。Apply 阶段在本仓库实现 CampusClaw 最小可运行栈。

Goals / Non-Goals

Goals:

- 登录可用，会话携带 `role` 与 `class_id`。
- 权限在服务端可判定（学生上传 403）。
- 班级隔离不可通过改前端绕过（查询与按 ID 访问均校验班级）。
- 教师上传链路：落盘 → 解析 → 双表入库 → 列表查库可见。
- `docker compose up` 一键启动；`GET /health` 供探活；密钥与 DB 路径可配置。

Non-Goals:

- 见 `proposal.md` Non-goals（检索问答、对话、作业、SSO、生产 HA 等）。

> PPT 点评
> design 回答“proposal 怎么落地”；Goals 与 spec 的 Requirement 一一呼应，Non-Goals 直接引用 proposal。

EXEMPLAR 03 / 10 · Ex.2

终稿 · design.md（中）

会话、栈与班级隔离 · 原文全文（Markdown 格式排版）

会话与密码

会话模型

- 采用 Flask 服务端会话（signed cookie）：登录成功后设置 HttpOnly、`SameSite=Lax` 的 session cookie。
- Session payload 至少含：`user_id`、`role`（`teacher` | `student`）、`class_id`。
- 签名密钥 `SECRET_KEY` 仅从环境变量读取（Compose `env_file: .env`）；禁止在源码中写死生产密钥。
- 受保护路由使用统一前置逻辑（装饰器或 `before_request`）：无有效 session → 页面 `302` 至 `/login`；API 返回 `401` JSON。

密码存储

- 用户表字段 `password_hash`（禁止 `password` 明文列）。
- 种子脚本与（若有）改密逻辑使用 bcrypt（或 argon2）；登录时用库提供的 verify（如 `werkzeug.security.check_password_hash` 或 `bcrypt.checkpw`）。
- 比较 SHOULD 使用恒定时间语义，避免时序侧信道（依赖库默认行为即可）。

登出

- `POST /logout` 或 `GET /logout` 清除 session，重定向登录页（实现细节，spec 未强制路径名）。

技术栈：Flask + SQLite

层次	选择	说明
Web	Flask 3.x	路由、模板 SSR 材料列表/登录页；上传与 JSON API 同进程
DB	SQLite 3	单文件 `data/app.db`，标准库 `sqlite3` 或薄封装
密码	bcrypt / werkzeug	`requirements.txt` 显式列出
运行	gunicorn（Compose 内）或 `flask run`（本地 dev）	Compose 生产式入口推荐 gunicorn 绑定 `0.0.0.0:8080`

备选栈： Node + Express + better-sqlite3 —— 行为等价即可，本 design 按 Python 栈写 tasks；若换栈须同步改 tasks 命令但 spec 不变。

模块划分（建议）

```
app/
  __init__.py      # create_app(), SECRET_KEY 校验
  auth.py          # login/logout, login_required
  materials.py     # 列表、按 id、upload
  knowledge.py     # parse_file + insert knowledge_entries
  db.py            # get_conn, 带 class_id 的查询 helper
  templates/       # login.html, materials.html
scripts/init_db.py
```

服务端班级隔离

隔离 MUST 实现在服务端，不得仅隐藏前端按钮。

列表与集合查询

- 所有 `SELECT` 材料/知识库条目时 强制 `WHERE class_id = :session_class_id`。
- 封装为 `list_materials(class_id)` 等函数，业务路由禁止传入客户端可控的 `class_id` 覆盖会话（管理员场景本 change 无）。

按 ID 访问单条资源

1. `SELECT` by `id`（可不带 class 条件以检测越权）。
2. 若行不存在 → `404`。
3. 若 `row.class_id != session['class_id']` → **`403`** 或 **`404`**（择一，在 README 固定），响应体不得含他班标题/正文/路径。

上传与写入

- 新记录的 `class_id` 只取自 session，忽略表单中的 class 字段（若有则丢弃或拒绝）。
- 教师仅能在本会话班级写入；不存在「代传 B 班」API。

测试关注点

- 学生 A1 列表不见 B 班标题；教师 A 用 B 班 material id 访问被拒；与 spec 三个班级 Scenario 一一对应。

> PPT 点评
>
> Flask 会话 + bcrypt；列表与按 ID 访问都要带 class 过滤——对应 spec 跨班 Scenario。

EXEMPLAR 04 / 10 · Ex.2

终稿 · design.md（下）

上传入库数据流 · 原文全文（Markdown 格式排版）

上传与入库数据流

```
Client (teacher, multipart/form-data)
  POST /api/materials/upload  或 POST /materials/upload
    |
    ├─ login_required + role == teacher  else 403
    ├─ 校验文件扩展名/大小（MVP: .txt, .md）
    ├─ 保存 uploads/{class_id}/{uuid}_{safe_filename}
    ├─ parse → plain text（失败则删文件，返回 400，无 DB 行）
    ├─ BEGIN TRANSACTION
    |     INSERT materials (title, class_id, file_path, uploaded_by, ...)
    |     INSERT knowledge_entries (material_id, class_id, body_text, ...)
    ├─ COMMIT
    └─ 201 { "material_id": ... }

Client GET /materials  (或 /api/materials)
    |
    ├─ login_required
    ├─ rows = list_materials(session.class_id)   # 仅本班
    └─ 200 HTML 或 JSON
```

- 标题：默认取自文件名或表单字段；须在本班列表可展示。
- 知识库：MVP 将解析全文或摘要写入 `body_text`；后续 RAG 不在本 change。
- 学生：同一 GET 列表 API，无 POST 上传路由权限。

> **PPT 点评**
>
> ASCII 流程图把事务边界写清了：解析失败就不写 DB 行，对应 spec 里“无脏数据”那条 Scenario。

EXEMPLAR 05 / 10 · Ex.2

终稿 · design.md（部署）

Docker Compose 与 /health · 原文全文（Markdown 格式排版）

Docker Compose 与 GET /health

文件约定

- `Dockerfile`：多阶段或单阶段安装依赖、复制 `app/`、`scripts/`；`CMD` 启动 gunicorn 或等价。
- `docker-compose.yml`：
  - service `app`，`build: .`，`ports: ["8080:8080"]`（或与 README 一致）。
  - `volumes`：`./data:/app/data`，`./uploads:/app/uploads`。
  - `env_file: .env`；`environment` 可覆盖 `SECRET_KEY`。
  - `healthcheck`：例如 `curl -f http://127.0.0.1:8080/health` 或 `wget -qO- ...`。
- `.env.example`：列出 `SECRET_KEY=` 占位与说明；不提交真实密钥。

启动与初始化

- 容器 entrypoint 或首次启动：若 `data/app.db` 不存在则运行 `python scripts/init_db.py`（建表 + 种子 A/B 班、用户、样本材料）。
- README 步骤：`cp .env.example .env` → 填 `SECRET_KEY` → `docker compose up --build` → 打开 `http://localhost:8080/login`。

GET /health

- 路由：`GET /health`，无需登录。
- 响应：`200`，`Content-Type: application/json`，body 如 `{"status":"ok"}`。
- 可选：检测 SQLite 文件可读；失败时返回 `503`（本 change 可简化为进程存活即 ok）。
- Compose `healthcheck` 依赖此端点，避免「容器 up 但应用未就绪」。

Decisions（摘要）

#	决策	备选
1	Flask + SQLite	Node + Express
2	Signed cookie session	JWT
3	bcrypt 密码哈希	PBKDF2-only
4	单库 + class_id 过滤	每班独立库
5	MVP 解析 txt/md	PDF 管道后置

Risks / Trade-offs

- [PDF 解析复杂] → 首版仅 txt/md；失败场景 spec 已要求无脏数据。
- [403 vs 404 跨班] → 在 README 与 tasks 验收中固定一种。
- [Flask dev server] → Compose 内用 gunicorn。

Migration Plan

从零起步：由 `init_db.py` 建表与种子，无旧数据需要迁移。实现顺序见 `tasks.md`（数据 → 登录 → 隔离 → 上传 → Compose → validate）。

Open Questions

（无。）

> **PPT 点评**
> 规约层交代了 volume、.env.example 与 healthcheck；栈确定为 Flask（不用 Node）。第 3 课 T6 按此实现。

EXEMPLAR 06 / 10 · Ex.3

终稿 · spec.md（上）

登录与角色 · 原文全文（Markdown 格式排版）

Purpose

让教师与学生以各自角色登录 CampusClaw，以班级作为数据边界隔离教学材料；教师上传的内容解析后写入知识库，本班成员通过列表查看；越权上传、跨班访问与未登录访问在服务端被拒绝。

ADDED Requirements

Requirement: 用户登录

系统 MUST 提供教师（teacher）、学生（student）两类角色的账号密码登录；登录成功后 MUST 建立服务端会话；未登录用户访问受保护页面 MUST 被引导到登录页，且 MUST NOT 泄露任何业务数据。

Scenario: 教师与学生登录成功

- WHEN 用户使用有效账号与密码发起登录（预置账号含教师 A、学生 A1、学生 B1）
- THEN 登录 MUST 成功并建立服务端会话
- AND 会话中 MUST 记录用户标识、角色（teacher 或 student）与所属班级（如班级 A 或 B）

Scenario: 错误密码登录失败

- WHEN 用户提供存在用户名但错误密码
- THEN 登录 MUST 失败
- AND 系统 MUST NOT 建立有效会话
- AND 响应 MUST NOT 暗示密码哪一位错误以外的敏感信息（如不返回密码哈希）

Scenario: 未登录访问受保护页面

- WHEN 未携带有效会话的用户请求受保护页面（如材料列表）或受保护 API
- THEN 对页面请求 MUST 重定向到登录页
- AND 对 API 请求 MUST 返回未授权（如 HTTP 401）
- AND 响应 MUST NOT 包含任何本班或他班材料标题、正文或文件路径

Requirement: 角色权限

系统 MUST 按会话中的角色授权：教师可上传与管理本班材料；学生对本班材料只读；学生调用上传或管理接口 MUST 被拒绝。

Scenario: 学生上传被拒绝（403）

- WHEN 以 student 角色（如学生 A1）的有效会话向材料上传接口提交文件
- THEN 系统 MUST 拒绝请求并返回 HTTP 403（或等价语义的错误码）
- AND `materials` 与知识库相关表 MUST 无新增或变更记录
- AND 上传目录 MUST 无因该请求产生的新文件（或事务回滚后不可见）

Scenario: 教师上传被允许

- WHEN 以 teacher 角色（如教师 A）的有效会话向材料上传接口提交支持的文件
- THEN 系统 MUST 接受请求并执行入库流程（见「材料上传与知识库入库」）
- AND 返回 MUST 表示成功（如 HTTP 201）并含新材料标识

Scenario: 学生仅只读本班列表

- WHEN 学生 A1 登录后打开本班材料列表
- THEN 系统 MUST 展示本班可读材料
- AND 页面或 API MUST NOT 提供学生可用的上传或删除本班材料的写操作入口（服务端仍 MUST 拒绝写 API，见上一 Scenario）

> **PPT 点评**
>
> WHEN/THEN/AND 都写成可判定的句子；“错误密码”“未登录 API 401”这类负路径与正路径同等重要。

EXEMPLAR 07 / 10 · Ex.3

终稿 · spec.md（中）

隔离与上传 · 原文全文（Markdown 格式排版）

Requirement: 班级隔离

班级是数据边界。所有材料、知识库及相关业务查询 MUST 在服务端按会话中的班级标识过滤；A 班用户 MUST NOT 读取、修改或删除 B 班材料；前端隐藏按钮 MUST NOT 作为满足本要求的唯一手段。

Scenario: 跨班按 ID 访问材料被拒绝

- WHEN 班级 A 的用户（学生 A1 或教师 A）通过 URL、API 路径参数或请求体指定 B 班某条材料的 ID 发起访问
- THEN 系统 MUST 拒绝访问（HTTP 403 或 404，实现择一并在文档中固定）
- AND 响应 MUST NOT 返回 B 班材料的标题、正文片段、文件路径或存储键

Scenario: 跨班枚举列表不得泄露 B 班

- WHEN 学生 A1 登录后请求材料列表（含分页、搜索参数为空或任意合法值）
- THEN 返回集合中每一条记录的班级归属 MUST 均为班级 A
- AND MUST NOT 出现 B 班预置材料的可区分标题（如标题中含「B 班」的样本）

Scenario: 材料列表仅含本班记录

- WHEN 教师 A 登录后打开本班材料列表
- THEN 列表中每一条记录 MUST 归属于班级 A
- AND 列表数据 MUST 来自数据库查询且查询条件含班级 A 的过滤

Requirement: 材料上传与知识库入库

教师上传教学材料后，系统 MUST 将文件落盘、解析文本（或等价内容）并写入知识库记录；材料与知识库条目 MUST 关联上传者所属班级；本班材料列表 MUST 从数据库查询展示；上传成功后刷新列表 MUST 可见新记录。

Scenario: 上传后知识库与列表可查

- WHEN 教师 A 通过上传接口提交一份合法文件（如 `.md` 或 `.txt`）且解析成功
- THEN 知识库（或 `knowledge_entries` 等价表）中 MUST 出现与该文件对应的新记录
- AND 该记录 MUST 关联班级 A 与对应材料主键
- AND 材料表 MUST 新增一条标题可辨的记录
- AND 教师 A 刷新本班材料列表时 MUST 看到该条新记录

Scenario: 上传后学生本班可见只读

- WHEN 教师 A 成功上传新材料后，学生 A1 刷新本班材料列表
- THEN 学生 A1 MUST 能在列表中看到该条新材料
- AND 学生 A1 对该材料 MUST 仍为只读（不得通过 API 修改或再次上传覆盖）

Scenario: 上传失败时不产生脏数据

- WHEN 上传过程因不支持格式、空文件或解析失败而终止
- THEN 系统 MUST 返回明确错误（如 HTTP 400）
- AND MUST NOT 在材料表或知识库中留下不完整记录
- AND MUST NOT 保留孤立的未关联文件（或 MUST 在失败时清理已写文件）

> **PPT 点评**
>
> 班级隔离写了三条 Scenario（按 ID、列表、教师列表）；上传一节还写了学生只读、且能看到新记录。

EXEMPLAR 08 / 10 · Ex.3

终稿 · spec.md（下）

预置、安全与部署 · 原文全文（Markdown 格式排版）

Requirement: 预置核心数据

系统 MUST 建立班级、用户、讲义、作业、助手、技能六类核心数据结构（表或等价实体），并预置可验收的样本数据，以支持登录、班级隔离与上传验收。

Scenario: 种子数据满足双班与用户

- WHEN 首次执行数据库初始化或种子脚本（含 Compose 首次启动时的 entrypoint）
- THEN 库中 MUST 存在班级 A、班级 B
- AND MUST 存在教师 A（归属班级 A）、学生 A1（归属 A）、学生 B1（归属 B）
- AND 各用户 MUST 具备可登录的用户名与已哈希的密码字段

Scenario: 两班材料标题可区分

- WHEN 种子脚本执行完成
- THEN MUST 存在至少一条明确归属 A 班的材料标题（如含「A 班」）
- AND MUST 存在至少一条明确归属 B 班的材料标题（如含「B 班」）
- AND 六类核心结构中讲义、作业、助手、技能表 MUST 已创建（可各含 0~1 条占位行，不要求本 change 验收业务功能）

Requirement: 密码哈希与会话密钥

系统 MUST 使用单向密码哈希算法存储用户密码，禁止明文；会话签名密钥（如 `SECRET_KEY`）MUST 仅通过服务端环境变量或 Compose 注入，MUST NOT 硬编码在源码或提交到版本库。

Scenario: 库中无明文密码

- WHEN 直接查询用户表中密码相关字段（如 `password_hash`）
- THEN 字段值 MUST NOT 等于任何已知预置账号的明文口令
- AND MUST 可识别为哈希格式（如 bcrypt `$2` 前缀或 argon2 标识）

Scenario: 登录校验使用哈希比较

- WHEN 用户使用正确明文密码登录
- THEN 系统 MUST 通过哈希验证通过并建立会话
- WHEN 同一用户使用错误密码登录
- THEN 验证 MUST 失败且不建立会话

Scenario: 缺少会话密钥时服务不得静默使用默认值

- WHEN 生产/Compose 配置未提供会话签名必需的环境变量
- THEN 应用启动 MUST 失败或使用文档明确禁止的不安全默认值；`.env.example` MUST 列出所需变量名

Requirement: Docker Compose 部署与健康检查

系统 MUST 以 Docker Compose 作为标准启动方式；应用 MUST 提供 `GET /health`；数据库文件与上传目录 MUST 通过 volume 持久化，以便容器重建后数据仍在。

Scenario: Compose 启动后可访问

- WHEN 操作者按 README 复制 `.env.example` 并执行 `docker compose up --build`（或文档等价命令）直至 healthcheck 通过
- THEN 浏览器 MUST 可访问登录页 URL
- AND `GET /health` MUST 返回 HTTP 200 且 body 表明服务可用（如 JSON `status: ok`）

Scenario: health 不依赖登录态

- WHEN 未携带会话 cookie 的请求访问 `GET /health`
- THEN MUST 仍返回成功响应
- AND MUST NOT 重定向到登录页

Scenario: 重建容器后数据仍在

- WHEN 已存在预置用户、材料或教师上传产生的记录后，执行 `docker compose down` 再 `docker compose up` 且未删除数据 volume
- THEN 预置班级与用户 MUST 仍可登录
- AND 已上传材料与知识库记录 MUST 仍可通过本班列表或 SQL 查询到

> **PPT 点评**
>
> 预置双班可分辨的标题、哈希密码、Compose/health/volume 三条 Scenario——第 2 课的验收口径到此闭合。

EXEMPLAR 09 / 10 · Ex.4

终稿 · tasks.md（上）

第 1–4 节 · 原文全文（Markdown 格式排版）

1. 仓库与文档（T1）

- ☐ 1.1 确认仓根 README 含价值/场景/不做三行说明 — verify: 打开 `README.md` 可见三行且与 proposal Non-goals 一致
- ☐ 1.2 添加 `requirements.txt`、应用入口（如 `run.py` 或 gunicorn 模块路径）与 `app/` 包骨架 — verify: `pip install -r requirements.txt` 后 `python -c "import app"` 无报错

2. 数据模型与种子（T2）

- ☐ 2.1 实现 `scripts/init_db.py`：创建班级、用户、讲义、作业、助手、技能、材料、知识库（或等价）表 — verify: 执行脚本后 `data/app.db` 存在且 `.schema` 含上述表名
- ☐ 2.2 种子：班级 A/B、教师 A、学生 A1/B1；A/B 班各至少一条可区分标题的材料；用户密码写入 `password_hash` — verify: `sqlite3 data/app.db "SELECT username, role FROM users"` 含 teacher_a、student_a1、student_b1
- ☐ 2.3 实现 `app/db.py`（连接、参数化查询、按 `class_id` 列表材料）— verify: 临时脚本对 class A 调用列表函数返回条数 ≥ 1 且不含 B 班标题

3. 登录、会话与密码（T3）

- ☐ 3.1 实现 `/login` 页面与 POST 登录逻辑；session 写入 `role`、`class_id`；`SECRET_KEY` 仅来自环境变量 — verify: 教师 A、学生 A1 登录后会话字段正确；未设置 `SECRET_KEY` 时启动行为与 design 一致
- ☐ 3.2 实现登出与 `login_required`：未登录访问 `/materials` 重定向登录页 — verify: 清 cookie 后 GET `/materials` 为 302 至 `/login` 且响应体无材料数据
- ☐ 3.3 密码 verify 走哈希；错误密码登录失败 — verify: 错误口令无法建立 session；库中 password 字段非明文

4. 班级隔离（T4）

- ☐ 4.1 材料列表 API/页面仅查询 `session.class_id` — verify: 学生 A1 登录后列表仅含 A 班标题，不出现 B 班样本标题
- ☐ 4.2 按 material id 访问时校验 `class_id` — verify: 教师 A 请求 B 班材料 id 返回 403 或 404（与 README 一致）且响应无 B 班正文/标题
- ☐ 4.3 确认路由不接受客户端传入的 `class_id` 覆盖会话 — verify: 篡改 query/body 中的 class 仍只能看到本班或越权被拒

> PPT 点评
>
> 顺序是先数据、再登录、后隔离；每条 verify 都能真的执行（sqlite、curl、302）。

EXEMPLAR 10 / 10 · Ex.4

终稿 · tasks.md（下）

第 5–7 节 · 原文全文（Markdown 格式排版）

5. 角色权限与上传入库（T5）

- ☐ 5.1 学生 POST 上传返回 403；DB 与 uploads 无新增 — verify: 学生 A1 上传后 `materials` / `knowledge_entries` 行数与上传前相同
- ☐ 5.2 教师上传：落盘 → 解析 txt/md → 同事务写入 materials + knowledge_entries，`class_id` 来自 session — verify: 教师 A 上传后两表各增 1 行且 class 为 A
- ☐ 5.3 上传成功后本班列表可见；学生 A1 刷新列表可见新条但仍无法上传 — verify: 教师上传后 A1 列表含新标题；A1 POST 上传仍 403
- ☐ 5.4 解析失败或非法扩展名返回 4xx 且无脏数据 — verify: 上传 `.exe` 或故意损坏文件后无孤立 DB 行

6. Docker Compose 与 GET /health（T6）

- ☐ 6.1 添加 `Dockerfile`、`docker-compose.yml`、`.env.example`；挂载 `./data`、`./uploads` — verify: `docker compose up --build -d` 后容器 running
- ☐ 6.2 实现 `GET /health` 无需登录；compose 配置 healthcheck — verify: `curl -sf http://localhost:8080/health` 返回 200 JSON
- ☐ 6.3 容器 entrypoint/启动时自动 init_db（若库不存在）— verify: 删 `data/app.db` 后 compose up 仍可登录预置账号
- ☐ 6.4 `docker compose down` 再 `up`（不删 volume）后预置与上传数据仍在 — verify: 上传一条后再 down/up，列表仍含该条
- ☐ 6.5 README 补充 Compose 启动步骤、登录 URL、health URL、`.env` 说明 — verify: 按 README 从零可打开登录页并 curl health

7. 规约与收尾

- ☐ 7.1 运行 `openspec validate add-auth-rbac-class-knowledge --strict` — verify: 命令 exit 0 且无 error
- ☐ 7.2 手工对照 spec 关键 Scenario：跨班被拒、学生上传 403、上传后列表可查 — verify: 记录三条验收结果（通过/失败）于 PR 或实验报告

> PPT 点评
>
> T5 上传、T6 Compose、T7 validate，再加三条 Scenario 手工对照——进 Apply 之前把规约收口。

PART III / REVIEW TEXT

4.6—4.9 Review 原文与附录

收录课件提供的 Proposal/Design 核对清单、手工修改提示词、Scenario 核验表、tasks 覆盖度、新对话验收及 OpenSpec 技能命令对照。此部分为教师原文；请将其中的 `<change>` 换成自己仓库的实际目录名。

REVIEW 01 / 05

4.6 · Review：proposal 与 design

4.6 Review：proposal 与 design

- 只允许修改 Markdown。顺序与官方一致：`proposal.md` → `specs/…/spec.md` → `tasks.md`。
- 与相邻同学交叉阅读：若对方无法从你的文件中读出“要做什么、不做什么”，则需要重写。

文件	与相邻同学对照（打勾自检）
README	三行说明写全：价值主张、核心场景、本学期不做
proposal.md	目标含登录、角色权限、班级隔离、上传入库；非目标含检索问答、对话、作业提交
design.md · 会话与密码	写清用会话还是 Token；密码哈希存储，禁止明文
design.md · 隔离与上传	班级过滤放在服务端；上传链路写到“文件 → 解析 → 入库 → 列表”
design.md · 部署节	写明 Docker Compose 启动、volume 持久化与 `GET /health`

手工改规约提示词（发现缺项时用）

```
请只修改 openspec/changes/<change>/ 下的 markdown，不要写代码。

我发现三处问题，请逐条修改：
1. design.md 没写班级隔离在哪一层实现。请补：所有业务查询在服务端按会话中的班级过滤，前端隐藏按钮不算满足。
2. proposal 的非目标不全。请补：注册与找回密码、技能开关。
3. spec 的班级隔离只有 Requirement、没有 Scenario。请补跨班访问被拒绝的 Scenario，用当 / 且 / 那么写。

改完告诉我每处改了哪个文件、改了什么。
```

REVIEW 02 / 05

4.7 · Review：spec 与 Scenario

4.7 Review：spec 与 Scenario

Requirement 主题	Scenario 必须能判定（打勾自检）
用户登录	教师、学生登录成功；未登录被引导到登录页
角色权限	学生调用上传返回 403；教师上传被允许
班级隔离	A 班用户读 B 班材料被拒；列表只含本班记录
上传与入库	上传后知识库有记录，刷新列表可见
六类核心数据	班级 A / B、教师 A、学生 A1 / B1、两班材料都能查到
Docker 部署	按 README 起 Compose 可访问；`/health` 返回成功；重建容器数据仍在

对照

唯一标准：Scenario 必须能够直接判定“通过”或“不通过”。若无法判定，说明尚未写完，请回到 4.5 补充。

REVIEW 03 / 05

4.8 · Review：tasks 覆盖度

4.8 Review：tasks 覆盖度

T1 · 规则与仓

规则文件、README 三行说明、四份规约均已就位——且不含任何代码。

T2–T4 · 登录与权限

先准备数据，再实现登录，再做隔离。顺序不可颠倒，且每条都必须可验证。

T5–T6 · 上传与 Docker

上传入库是核心环节——教师可上传、学生被拒绝。T6 的 Compose 与 `/health` 属于第 3 课。

本段结束标准

此处自查：tasks 完整覆盖 spec 的每一条要求，checkbox 全部为 `[ ]`。

REVIEW 04 / 05

4.9 • 新对话验收：规约已进仓

4.9 新对话验收：规约已进仓

- 关闭此前用于提案的对话，新建一个会话。切勿把之前的聊天记录粘贴过去。
- 观察它能否独立读懂仓库，准确说出硬性要求，并指出每条要求的依据文件。

新对话验收提示词

```
不要猜。请阅读本仓库的 openspec/（含 changes 与 specs）、README.md 与项目规则文件。
1. 列出本变更的硬性要求（至少 3 条），每条给出对应的文件路径。
2. A 班的学生能否查看 B 班的材料？请引用 spec 原文说明。
3. 教师上传材料后，知识库与材料列表分别怎么变化？请引用 spec 原文。
不要改任何代码。
```

追问提示词（回答没引用原文时用）

```
你刚才的回答没有引用 spec 原文。请打开 openspec/changes/<change>/specs/ 下的 spec.md，
把“班级隔离”“材料上传与知识库入库”两个 Requirement 的 Scenario 原文贴出来，
再据此回答：跨班访问会被拒绝吗？依据是哪一条 Scenario？
```

要点

更换会话后依然能够复现要求，这正是把规约写进仓库的意义。

REVIEW 05 / 05

附录 · OpenSpec 技能与斜杠命令对照

附录：OpenSpec 技能与斜杠命令对照

要做的事	技能名	Claude 常见斜杠命令
Explore（可选）	openspec-explore	思考模式，不写文件
提案	openspec-propose	/opsx-propose
实现（课后）	openspec-apply-change	/opsx-apply CHANGE
改计划	openspec-update-change	/opsx-update（以 init 提示为准）
归档	openspec-archive-change	/opsx-archive

本课禁止

本课“三不”：不 Apply、不编写业务页面、不运行真实 Compose。此外：课表练习仓不计入本次交付；规约变更仅限于 `openspec/**/*.md`，口头约定无效。

ENDNOTES / 阅读提示

原文说明与技术勘误提示

1. 原句保留：Ex.3“缺少会话密钥时服务不得静默使用默认值”Scenario 原文包含“应用启动 MUST 失败或使用文档明确禁止的不安全默认值”。该表达可能造成双重解读，实施时建议以同段 Requirement 的“不得硬编码或使用不安全默认值”为准，并与教师确认；本册和附带的原文 Markdown 均未代改课件原句。

2. 术语提示：Ex.2 写“Flask 服务端会话（signed cookie）”。Flask 默认 session 是客户端持有的签名 Cookie，不应误以为登录数据默认保存在服务器数据库。该术语疑点仅作为阅读提示，原文仍按课件保留。

3. 本课与后续作业：本册只针对第 2 课 SDD/OpenSpec。你之前提供的 F12 抓包、上传下载权限、Docker/Nginx 配置及系统运行截图是另一些平台作业，不应当作第 2 课现阶段要求运行或实现的内容。

4. 原始来源：北京大学《第 2 课 · SDD 与 CampusClaw 规约》在线课件。Ex.1—Ex.4 与 Review 部分按本次对话中你贴出的课件内容排版。